Task

Final Project: Online Store Order Management

Course: Spring Data Access, Spring ORM JPA Integration **Type:** Individual capstone project **Suggested duration:** 2 weeks (see milestones) **Total:** 100 points (+ up to 10 bonus)

---

1. Scenario

You are building the **back end** of a small online store. The system must manage customers, products, orders and payments, and it must stay **consistent** when several things happen at once (for example, two customers buying the last item in stock).

There is no web layer. The application is driven by a `Main` class and by automated tests.

---

2. Technical Constraints (Read Carefully)

> ⚠ **Spring ORM + JPA integration ONLY The point of this project is to master the manual Spring/JPA integration.**

You MUST use

- `LocalContainerEntityManagerFactoryBean` with `HibernateJpaVendorAdapter`
- `JpaTransactionManager`
- `@EnableTransactionManagement` and `@Transactional`
- `@PersistenceContext EntityManager` inside hand-written `@Repository` classes
- `PersistenceExceptionTranslationPostProcessor`
- Java-based configuration (`@Configuration` , `@ComponentScan`)
- JPA annotations (`jakarta.persistence.*`), **JPQL** and the **Criteria API**

You MUST NOT use (marks are deducted)

- Spring Data JPA (`JpaRepository` , `@EnableJpaRepositories` , derived queries)
- `JdbcTemplate` or any Spring JDBC class for data access
- The native Hibernate API (`Session` , `SessionFactory` , HQL-only features)
- Spring Boot (plain Spring only)

---

3. Required Project Structure

```
store/
├── pom.xml
```

```
├── README.md
└── src
    ├── main/java/com/store
    │   ├── config/JpaConfig.java
    │   ├── model/        (entities, embeddables, enums)
    │   ├── dto/          (query result DTOs)
    │   ├── repository/   (one @Repository per aggregate)
    │   ├── service/      (business logic + @Transactional)
    │   ├── exception/    (custom business exceptions)
    │   └── Main.java     (demo scenario)
    └── test/java/com/store   (JUnit tests)
```

---

4. Part 1: Configuration

Create `JpaConfig` with:

1. A `DataSource` bean (embedded H2, unique name).
2. `LocalContainerEntityManagerFactoryBean`
   - `setPackagesToScan("com.store.model")`
   - `hibernate.hbm2ddl.auto=create-drop`
   - `hibernate.show_sql=true`
   - `hibernate.generate_statistics=true` (used in Part 6)
3. `JpaTransactionManager`
4. A **static** `PersistenceExceptionTranslationPostProcessor` bean

Checkpoint: the context starts and the tables are created (show the SQL log).

---

5. Part 2: Domain Model

Model the following. Every relationship must have a clear **owning side**, and bidirectional associations must be kept in sync with helper methods (`addItem` , `removeItem` , and so on).

Entity	Fields	Notes
`BaseEntity` (`@MappedSuperclass`)	`id` , `createdAt` , `@Version version`	All entities extend it
`Customer`	`name` , `email` (unique), `@Embedded Address shippingAddress` , `Set<Order> orders`	Address is `@Embeddable` (street, city, country)
`Category`	`name` (unique)	
`Product`	`sku` (unique), `name` , `price` (`BigDecimal`), `stock` (int), `Set<Category> categories`	`@ManyToMany` with Category
`Order`	`customer` , `status` (`@Enumerated STRING`), `orderedAt` , `List<OrderItem> items` , `Payment payment`	Table name **`orders`** (avoid the SQL keyword)
`OrderItem`	`order` , `product` , `quantity` , `unitPrice`	Price is copied from the product at order time

Entity	Fields	Notes
`Payment`	`order`, `amount`, `method` (enum), `paidAt`	`@OneToOne` with Order

Enums: `OrderStatus { NEW, PAID, SHIPPED, CANCELLED }`, `PaymentMethod { CARD, CASH, WALLET }`

Mapping requirements

- `Order` to `OrderItem`: `cascade = ALL`, `orphanRemoval = true`
- Every `@ManyToOne` and collection is `LAZY`
- `Order.getTotal()` calculated from the items (not stored)
- `equals/hashCode` on `Product` based on `sku`

---

6. Part 3: Repositories

Write one hand-written `@Repository` class per aggregate using `@PersistenceContext`. Do **not** put business rules here.

`CustomerRepository`

- `save`, `findById`, `findByEmail(String)` (returns `Optional`), `findAll()`

`ProductRepository`

- `save`, `findById`, `findBySku(String)`
- `findByCategory(String categoryName)`
- `search(String keyword, BigDecimal minPrice, BigDecimal maxPrice, String category)` using the **Criteria API**, with every filter optional
- `findLowStock(int threshold)`
- `findPage(int page, int size)` with a total count method `countAll()`

`OrderRepository`

- `save`, `findById`
- `findByIdWithItems(Long id)` using `join fetch` (avoid N+1)
- `findByCustomer(Long customerId)`
- `findByStatus(OrderStatus status)`

`ReportRepository` (Part 5)

---

7. Part 4: Services and Business Rules (25 pts)

Services own the transactions. Use custom **unchecked** exceptions in `com.store.exception`.

`OrderService`

Method	Rules
`placeOrder(customerId, Map<Long,Integer> productQuantities)`	Creates a `NEW` order. For each product: quantity must be greater than 0, and stock must be sufficient, otherwise throw `InsufficientStockException`. Decrease stock. Copy the current price into `OrderItem`. **All-or-nothing:** one bad line rolls back the whole order.
`pay(orderId, PaymentMethod)`	Only a `NEW` order can be paid (otherwise `InvalidOrderStateException`). Creates the `Payment`, and sets status to `PAID`.
`ship(orderId)`	Only a `PAID` order can be shipped.
`cancel(orderId)`	Allowed for `NEW` or `PAID` only. **Restores stock** for every item.
`getOrderSummary(orderId)`	`readOnly = true`. Returns a DTO (no lazy-loading exceptions).

`CustomerService`

- `register(name, email, Address)`: reject duplicate emails with `DuplicateCustomerException`.

`ProductService`

- `addProduct(...)`, `restock(productId, quantity)`, `changePrice(productId, newPrice)`

Transaction requirements

- Use `@Transactional` on service methods only, never on repositories.
- Read-only methods use `readOnly = true`.
- Demonstrate `Propagation.REQUIRES_NEW` in at least one place with a valid reason (for example, an audit-log entry `AuditLog` that must be saved even if the main operation fails).
- Show that `pay()` **does not** need an explicit `save` call for the status change (dirty checking).

---

8. Part 5: Queries and Reports (10 pts)

Implement in `ReportRepository`, returning DTOs (Java `record`s) through **JPQL constructor expressions** unless stated otherwise.

1. `revenueByCategory()` returns `List<CategoryRevenue(String category, BigDecimal revenue)>` (only `PAID` and `SHIPPED` orders)
2. `topCustomers(int limit)` returns `List<CustomerSpend(String name, BigDecimal total)>`, highest first

3. `ordersPerStatus()` returns `Map<OrderStatus, Long>`
4. `productsNeverOrdered()` using `not exists` or `left join ... is null`
5. `monthlySales(int year)` returns `List<MonthlySales(int month, BigDecimal total)>`
6. **Bulk update:** `applyDiscount(String category, double percent)` reduces prices with a single JPQL update, then clears the persistence context. Explain in the README why `clear()` is needed.
